

Chapter 8 연습문제 과제

김수민

2026-05-03

연습문제 2: 단순 지수 평활(SES) 함수 직접 구현

단순 지수 평활을 수행하는 함수를 직접 작성하고, ETS() 함수의 결과와 비교합니다.

```
#
my_ses <- function(y, alpha, level) {
  n <- length(y)
  y_hat <- numeric(n + 1)
  y_hat[1] <- level #
  for (i in 1:n) {
    y_hat[i + 1] <- alpha * y[i] + (1 - alpha) * y_hat[i]
  }
  return(y_hat[n + 1]) # (1 )
}

# aus_livestock ( )
pigs <- aus_livestock |>
  filter(State == "Victoria", Animal == "Pigs")

# ETS()
fit_ets <- pigs |>
  model(ETS(Count ~ error("A") + trend("N") + season("N")))

# ETS alpha level
tidy_ets <- tidy(fit_ets)
alpha_opt <- tidy_ets |>
  filter(term == "alpha") |>
  pull(estimate)
level_opt <- tidy_ets |>
  filter(term == "l[0]") |>
  pull(estimate)

#
my_forecast <- my_ses(pigs$Count, alpha_opt, level_opt)

# ETS 1
ets_forecast <- fit_ets |>
  forecast(h = 1) |>
  pull(.mean)
```

```
data.frame(
  Method = c("my_ses()", "ETS()"),
  Forecast = c(my_forecast, ets_forecast)
) |> knitr::kable(caption = " ")
```

Table 1: 예측값 비교

Method	Forecast
my_ses() 함수	95186.56
ETS() 함수	95186.56

□ 해석: 직접 구현한 my_ses() 함수에 ETS()에서 구한 α 와 ℓ_0 를 입력하여 예측한 결과가 내장된 ETS() 함수의 1단계 앞 예측 결과와 완전히 일치하는 것을 확인할 수 있습니다.

연습문제 4: 최적 파라미터 탐색 및 예측 결합

위에서 만든 함수들을 결합하여, 오차 제곱합(SSE)을 최소화하는 최적의 α 와 ℓ_0 를 찾고 동시에 다음 시점을 예측하는 함수를 구현합니다.

```
# (SSE)
# [ ]      y[1] (y[1] - level) SSE
my_ses_sse <- function(params, y) {
  alpha <- params[1]
  level <- params[2]
  n <- length(y)
  # y_hat[i] y[i] 1
  # y_hat[1] = level(= ) y[1]
  y_hat <- numeric(n)
  y_hat[1] <- level
  for (i in 2:n) {
    y_hat[i] <- alpha * y[i - 1] + (1 - alpha) * y_hat[i - 1]
  }
  sse <- sum((y - y_hat)^2) # n SSE
  return(sse)
}

#
my_ses_full <- function(y) {
  # : alpha = 0.5, level =
  init_params <- c(0.5, y[1])

  # optim SSE
  # alpha 0 1 L-BFGS-B
  opt <- optim(
    par = init_params,
    fn = my_ses_sse,
    y = y,
    method = "L-BFGS-B",
    lower = c(0, -Inf),
```

```

    upper = c(1, Inf)
  )

  alpha_opt <- opt$par[1]
  level_opt <- opt$par[2]

  #
  forecast_val <- my_ses(y, alpha_opt, level_opt)

  list(
    alpha = alpha_opt,
    level = level_opt,
    forecast = forecast_val
  )
}

#      pigs
my_full_result <- my_ses_full(pigs$Count)

data.frame(
  Parameter = c("alpha", "level", "forecast"),
  `my_ses_full()` = c(my_full_result$alpha, my_full_result$level, my_full_result$forecast),
  `ETS()` = c(alpha_opt, level_opt, ets_forecast)
) |> knitr::kable(caption = "      ETS()      ")

```

Table 2: 직접 최적화한 결과와 ETS() 결과 비교

Parameter	my_ses_full..	ETS..
alpha	3.219966e-01	3.221247e-01
level	1.005329e+05	1.006466e+05
forecast	9.518674e+04	9.518656e+04

□ 해석: R의 `optim()` 함수를 활용하여 직접 구현한 최적화 결과와 내장 함수 `ETS()`의 추정치가 매우 유사하게 도출되었습니다. 극히 미세한 차이는 두 방식이 사용하는 최적화 알고리즘의 내부적인 허용 오차나 초기값 차이 등에서 기인한 것입니다.

연습문제 6: 중국 GDP 예측 (global_economy)

`global_economy` 데이터셋에서 중국의 GDP를 추출하여 다양한 옵션의 ETS 모형으로 예측하고 결과를 비교합니다.

```

china_gdp <- global_economy |>
  filter(Country == "China") |>
  select(Year, GDP)

#      ETS,      (Damped trend), Box-Cox      ETS
fit_china <- china_gdp |>
  model(
    ets      = ETS(GDP),

```

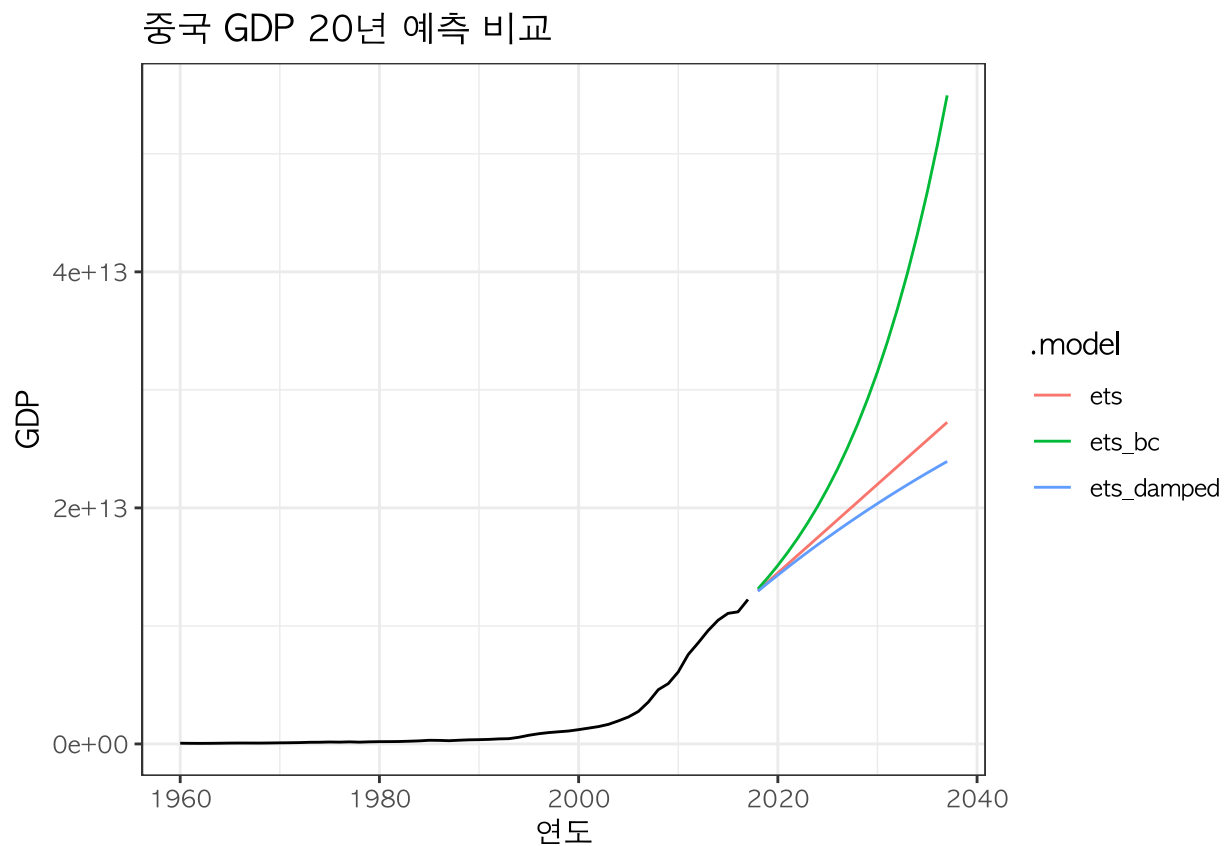
```

ets_damped = ETS(GDP ~ trend("Ad")),
ets_bc      = ETS(box_cox(GDP, 0.2)) # lambda = 0.2
)

# h=20
fc_china <- fit_china |> forecast(h = 20)

fc_china |>
  autoplot(china_gdp, level = NULL) +
  labs(title = " GDP 20 ", x = " ", y = "GDP") +
  theme_bw(base_family = "AppleGothic")

```



□ 해석 및 직관:

- 자동 ETS 모델 (**ets**): 최근 데이터의 가파른 GDP 성장세를 강한 선형 또는 지수적 추세로 인식하여, 미래에도 멈추지 않고 끝없이 폭발적으로 성장할 것이라고 예측합니다.
- 감쇠 추세 모델 (**ets_damped**): 강한 성장세가 영원히 지속될 수 없다는 현실적 가정(감쇠 추세)을 반영하여, 시간이 지날수록 성장의 기울기가 점차 완만해지도록 예측합니다.
- Box-Cox 변환 모델 (**ets_bc**, $\lambda = 0.2$): 분산이 수준에 비례하여 증가하는 비선형적 성장 패턴을 완화한 후 모델링을 진행합니다. $\lambda = 0.2$ 는 로그 변환에 가까운 강한 변환으로, 역변환 시 다른 모델들보다 완만한 곡선 궤적을 그리게 됩니다.
- 장기 경제 성장 예측에서는 **ets_damped**와 같이 성장이 점진적으로 둔화됨을 가정하는 것이 직관적으로도, 현실적으로도 가장 안전하고 타당한 접근입니다.

연습문제 8: 소매 매출액 예측 (aus_retail)

과거 연습문제에서 사용했던 소매 시계열 데이터를 바탕으로 Holt-Winters 방법론을 적용하고 평가합니다.

```
# [ ]      Series ID      (set.seed + sample
#      Series ID      )
myseries <- aus_retail |>
  filter(`Series ID` == "A3349399C")

#
cat("  Series ID:", unique(myseries$`Series ID`), "\n")
```

```
##    Series ID: A3349399C
```

```
cat("  :", unique(myseries$Industry), "\n")
```

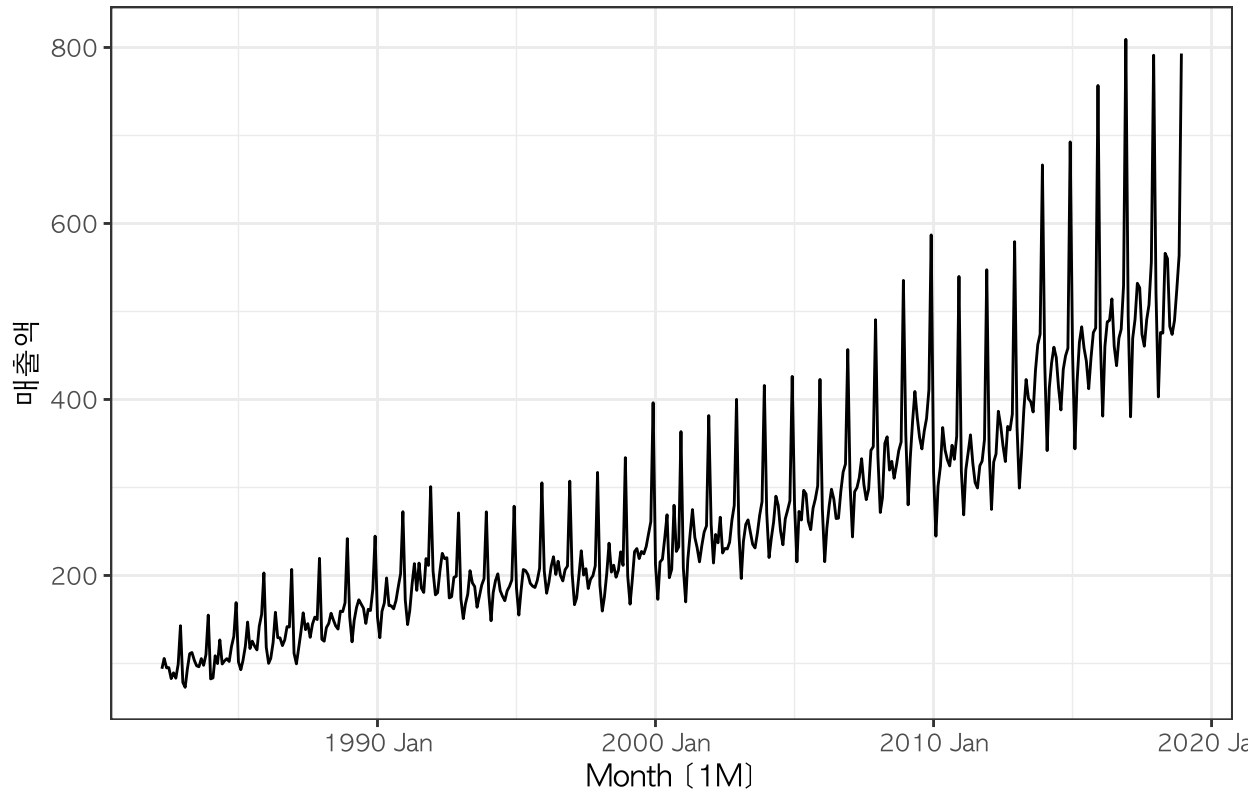
```
##   : Clothing retailing
```

```
cat(" (State):", unique(myseries$State), "\n")
```

```
## (State): New South Wales
```

```
# (a)
myseries |> autoplot(Turnover) +
  labs(title = "      ", y = " ") +
  theme_bw(base_family = "AppleGothic")
```

소매 매출액 시계열 플롯



(a) 곱셈적(Multiplicative) 계절성의 필요성: 위 그래프를 보면, 시간이 지나면서 전반적인 매출 수준이 증가함에 따라 계절적 변동의 진폭(파동의 위아래 폭)도 함께 비례하여 커지는 전형적인 확산 패턴을 보여줍니다. 이렇게 계절성의 크기가 추세 수준에 의존적일 때는 덧셈적 방식보다 곱셈적 계절성을 사용하는 것이 데이터의 분산을 안정화시키는 데 필수적입니다.

```
# (b) & (c) Holt-Winters (M,A,M) (M,Ad,M)
fit_hw <- myseries |>
  model(
    hw_multi = ETS(Turnover ~ error("M") + trend("A") + season("M")),
    hw_damped = ETS(Turnover ~ error("M") + trend("Ad") + season("M"))
  )

# (c) 1-step RMSE
hw_accuracy <- accuracy(fit_hw) |>
  select(.model, RMSE) |>
  arrange(RMSE)

hw_accuracy |>
  knitr::kable(caption = " (1-step) RMSE ")
```

Table 3: 모델별 훈련 데이터(1-step) RMSE 비교

.model	RMSE
hw_multi	13.66468

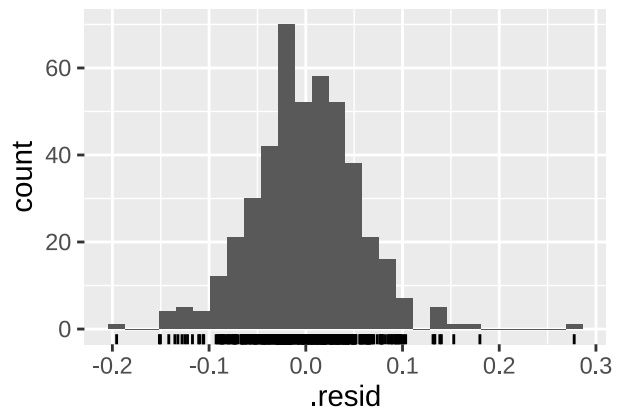
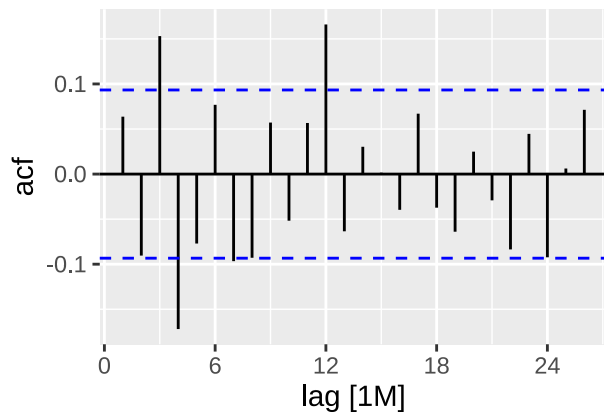
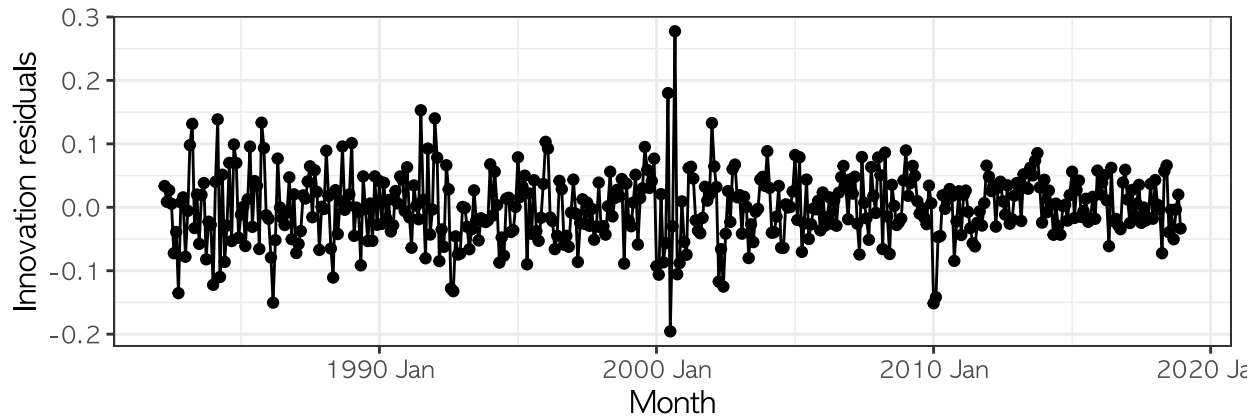
.model	RMSE
hw_damped	13.81248

```
#
best_model <- hw_accuracy$.model[1]
cat("\n (RMSE ):", best_model, "\n")
```

```
##
## (RMSE ): hw_multi
```

(c) 선호 모델 선택: 위 표에서 RMSE가 더 낮은 모델이 훈련 데이터(In-sample) 적합도 기준의 선호 모델입니다. 단기 예측에서는 선형 추세 모델(hw_multi)이, 장기 예측에서는 현실성을 고려한 hw_damped 모델이 유리합니다. 이하 잔차 분석은 RMSE가 더 낮은 모델을 기준으로 진행합니다.

```
# (d)
fit_hw |>
  select(all_of(best_model)) |>
  gg_tsresiduals() +
  theme_bw(base_family = "AppleGothic")
```



```
# Ljung-Box
fit_hw |>
  select(all_of(best_model)) |>
```

```
augment() |>
features(.innov, ljung_box, lag = 24) |>
knitr::kable(caption = "Ljung-Box  ")
```

Table 4: Ljung-Box 검정 결과

.model	lb_stat	lb_pvalue
hw_multi	75.67755	3e-07

(d) 잔차 확인: 잔차 플롯을 보면 대략 0을 중심으로 일정하게 분포하는 것으로 보입니다. 그러나 ACF 플롯과 Ljung-Box 검정(p-value가 매우 작음)의 결과를 보면, 계절성 시차 부근 등에서 약간의 유의미한 자기상관이 아직 남아있는 것을 확인할 수 있습니다. 따라서 이 모델의 잔차는 완벽한 백색 잡음(White noise)이라고 보기는 어려우며, 개선의 여지가 남아있습니다.

```
# (e) 2010
train <- myseries |> filter(year(Month) <= 2010)

fit_test <- train |>
  model(
    hw_multi = ETS(Turnover ~ error("M") + trend("A") + season("M")),
    snaive = SNAIVE(Turnover)
  )

fc_test <- fit_test |>
  forecast(new_data = anti_join(myseries, train, by = "Month"))

accuracy(fc_test, myseries) |>
  select(.model, RMSE) |>
  arrange(RMSE) |>
  knitr::kable(caption = "          RMSE  ")
```

Table 5: 테스트 세트 예측 RMSE 비교

.model	RMSE
hw_multi	77.95347
snaive	125.38306

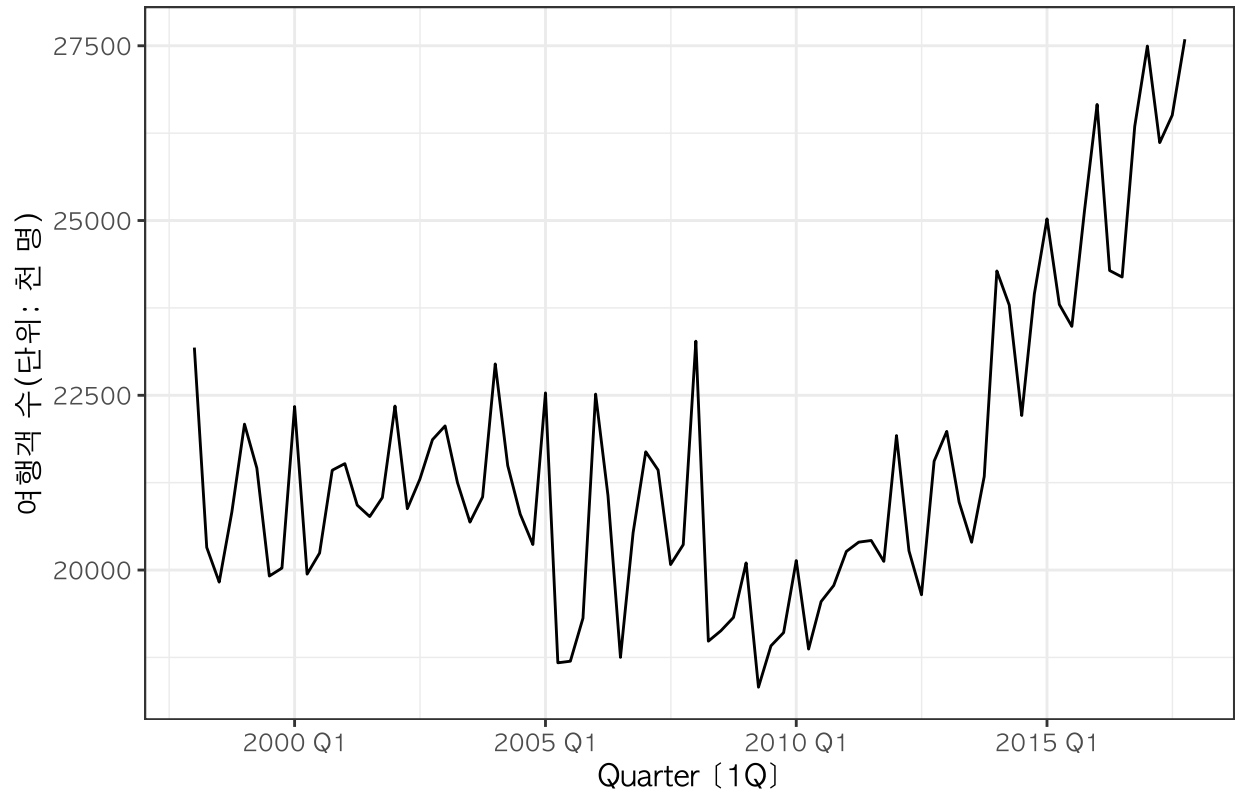
(e) Seasonal Naive와의 비교: 테스트 데이터셋에 대한 예측 RMSE를 비교해본 결과, 추세와 계절성 성분을 모두 동적으로 반영하는 Holt-Winters 모형(hw_multi)이 오직 작년 동기 데이터만을 그대로 가져오는 Seasonal Naive 모형(snaive)보다 예측 오차(RMSE)가 훨씬 적어 뛰어난 예측 성능을 보여주었습니다.

연습문제 10: 호주 내국인 여행객 수 (tourism)

```
aus_trips <- tourism |>
  summarise(Trips = sum(Trips))
```

```
# (a)
aus_trips |> autoplot(Trips) +
  labs(title = "(a) ", y = " ( : )") +
  theme_bw(base_family = "AppleGothic")
```

(a) 호주 내국인 여행객 수 총합



(a) 시계열 주요 특징: 매년 1분기와 4분기에 수치가 크게 상승하는 뚜렷한 계절성을 띠고 있습니다. 추세 측면에서는 2000년대 내내 어느 정도 정체된 박스권 움직임을 보이다가, 2010년 이후부터는 꾸준히 상승하는 우상향 추세로 전환된 특징을 보여줍니다.

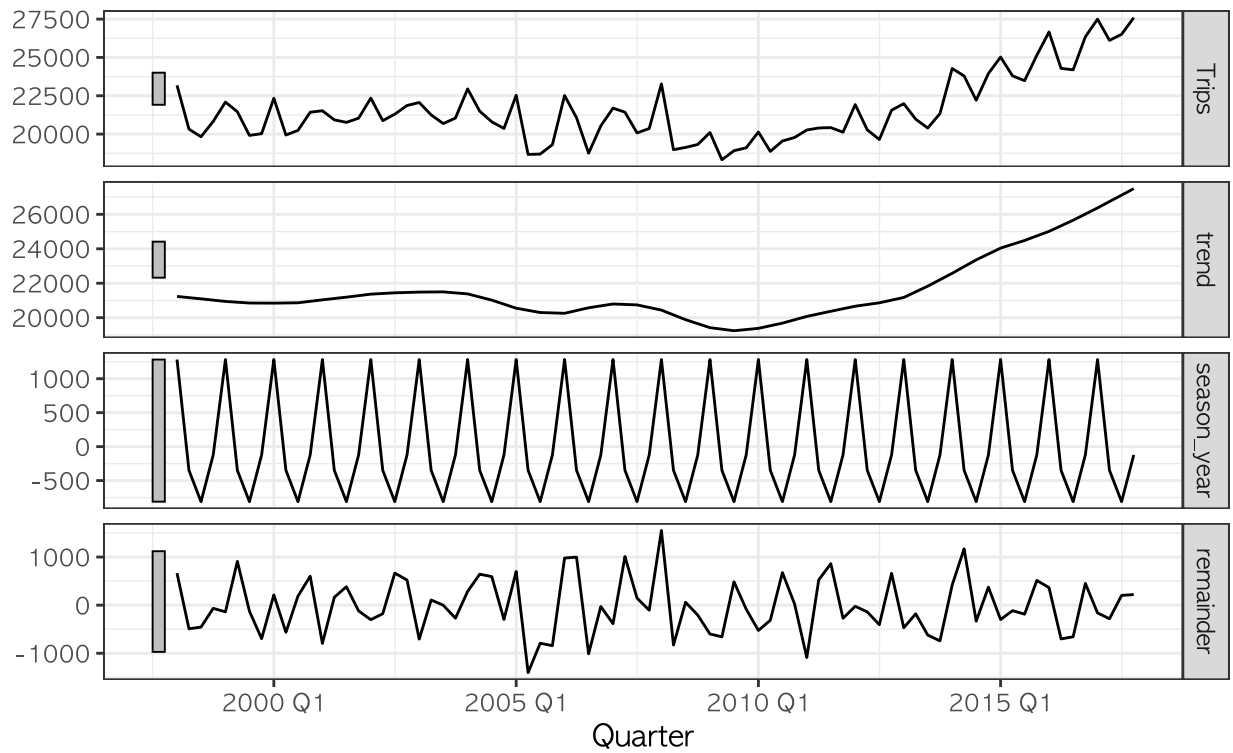
```
# (b) STL
stl_fit <- aus_trips |>
  model(STL(Trips ~ trend(window = 13) + season(window = "periodic")))

stl_components <- components(stl_fit)

#
stl_components |>
  autoplot() +
  labs(title = "(b) STL ") +
  theme_bw(base_family = "AppleGothic")
```

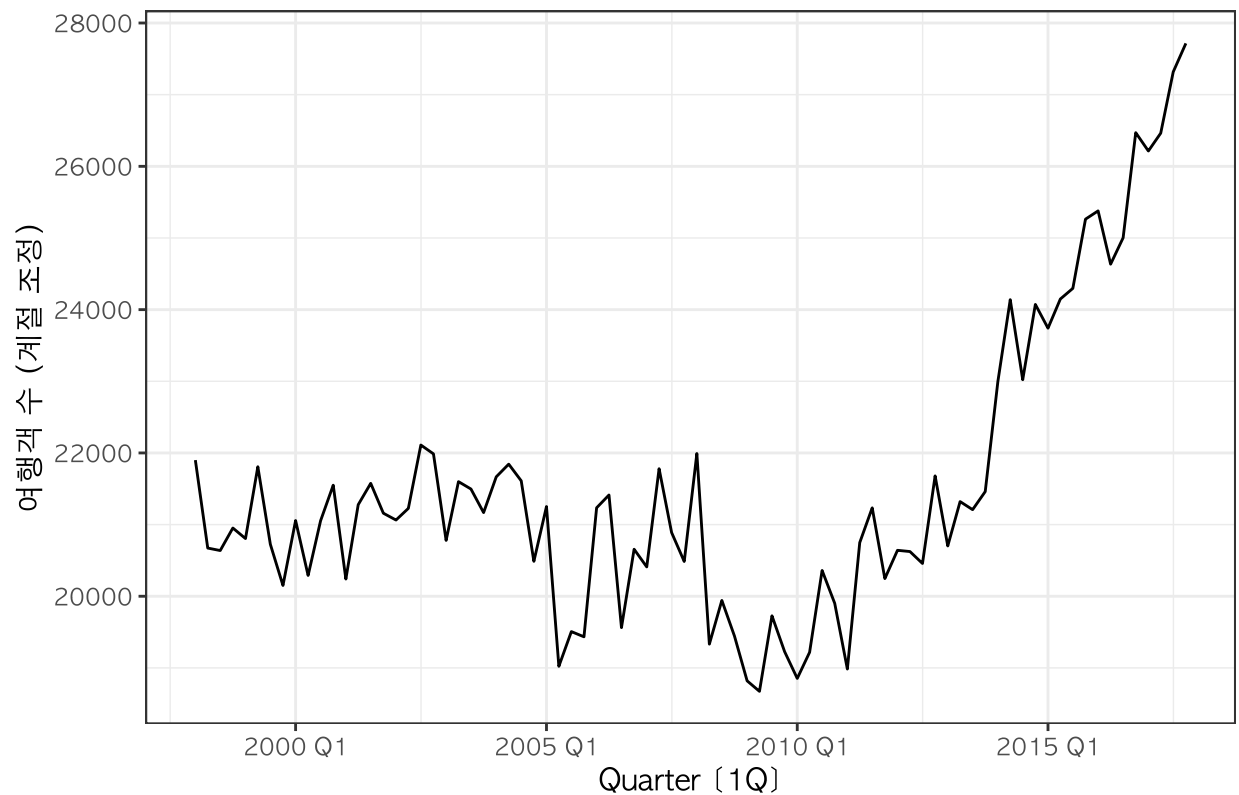
(b) STL 분해 결과

Trips = trend + season_year + remainder



```
# ( - )  
stl_components |>  
  select(Quarter, season_adjust) |>  
  autoplot(season_adjust) +  
  labs(title = "(b) ", y = " ( )") +  
  theme_bw(base_family = "AppleGothic")
```

(b) 계절 조정된 여행객 수



```
# (c), (d), (e)
fit_trips <- aus_trips |>
  model(
    # (c) (Ad)
    stl_damped = decomposition_model(
      STL(Trips ~ trend(window = 13) + season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("Ad") + season("N"))
    ),
    # (d) Holt (A)
    stl_linear = decomposition_model(
      STL(Trips ~ trend(window = 13) + season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("N"))
    ),
    # (e) ETS
    ets_auto = ETS(Trips)
  )

# (f) (In-sample) (RMSE)
accuracy(fit_trips) |>
  select(.model, RMSE) |>
  arrange(RMSE) |>
  knitr::kable(caption = "(f) RMSE ")
```

Table 6: (f) 각 모델별 훈련 데이터 RMSE 비교

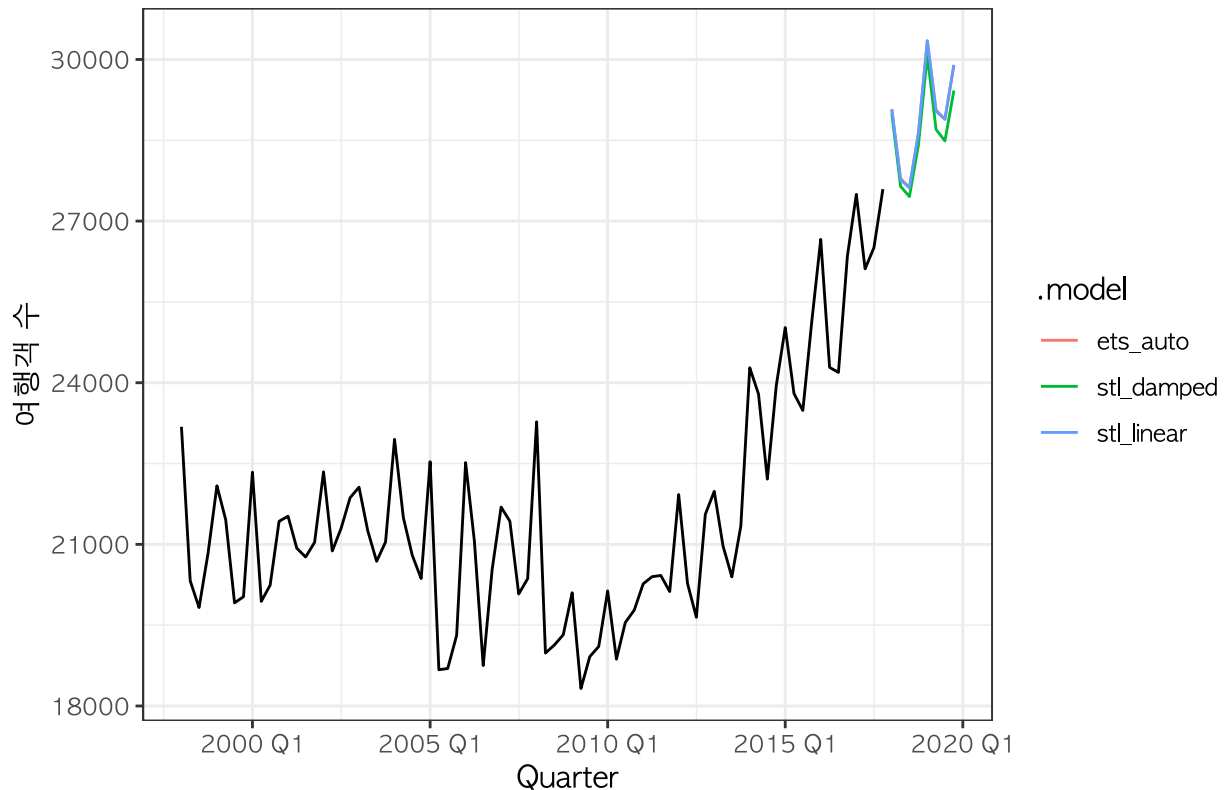
.model	RMSE
ets_auto	793.6695
stl_linear	800.6264
stl_damped	800.9873

(f) 적합도 비교: ets_auto 모델의 훈련 데이터 RMSE가 STL 분해를 거친 두 모형들보다 낮습니다. 즉, 전체 데이터를 한꺼번에 모델링하여 파라미터를 추정하는 자동 ETS 모형이 인샘플 데이터를 가장 잘 설명하고 있습니다.

```
# (g) 2
fc_trips <- fit_trips |> forecast(h = "2 years")

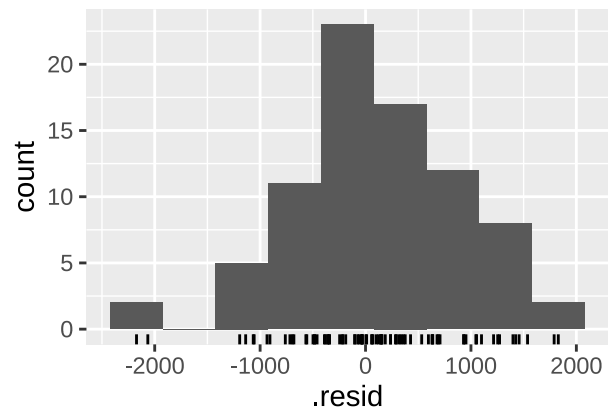
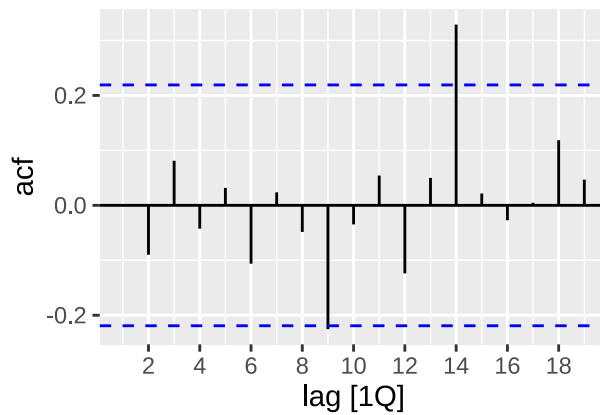
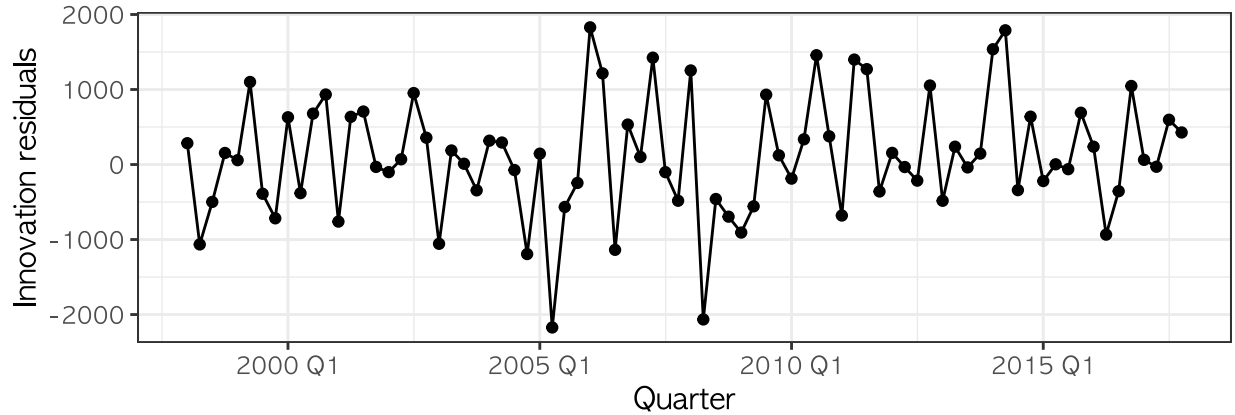
fc_trips |>
  autoplot(aus_trips, level = NULL) +
  labs(title = "(g) 2", y = " ") +
  theme_bw(base_family = "AppleGothic")
```

(g) 향후 2년 여행객 수 예측 비교



(g) 예측치 비교 및 타당성: 세 모형 모두 데이터에 내재된 계절 패턴을 훌륭하게 잡아냅니다. 차이는 향후 2년의 추세 예측에서 나타납니다. stl_linear는 최근의 가파른 상승세를 무한히 이어가고, stl_damped는 상승세가 다소 꺾일 것으로 예측합니다. ets_auto는 그 중간쯤의 현실적이고 완만한 상승세를 반영하고 있습니다. 실무적으로 무한한 선형 성장은 비현실적일 때가 많으므로, ets_auto나 stl_damped 모형의 예측 궤적이 가장 합리적(Reasonable)으로 판단됩니다.

```
# (h)
fit_trips |>
  select(ets_auto) |>
  gg_tsresiduals() +
  theme_bw(base_family = "AppleGothic")
```



```
# Ljung-Box
fit_trips |>
  select(ets_auto) |>
  augment() |>
  features(.innov, ljung_box, lag = 8) |>
  knitr::kable(caption = "(h) ets_auto Ljung-Box ")
```

Table 7: (h) ets_auto 모델 Ljung-Box 검정 결과

.model	lb_stat	lb_pvalue
ets_auto	2.743219	0.9494245

(h) 잔차 진단: 가장 선호하는 ets_auto 모델의 잔차를 진단해본 결과, 시간 플롯에서 잔차가 0 주위로 일정한 분산을 가지며 흩어져 있고 히스토그램도 대략 정규분포의 형태를 띕니다. ACF 플롯에서도 대부분의 시차가 파란 점선(유의 수준) 안에 들어가 있어 백색 잡음 가정에 크게 위배되지 않는 안정적인 모형임을 보여줍니다.

연습문제 12: 시계열 교차 검증 (Cross-validation)

시멘트 생산량 데이터(australian_cement_production)를 대상으로 시계열 교차 검증을 수행하여 ETS 모델과 Seasonal Naive 모델의 1년(4분기) 앞 예측 성능을 평가합니다.

(a) 교차 검증 설정 및 모델 적합

```
cement <- aus_production |> select(Quarter, Cement)

# 5 (20 )      1      CV
cement_cv <- cement |>
  stretch_tsibble(.init = 20, .step = 1)

#
fit_cv <- cement_cv |>
  model(
    ets = ETS(Cement),
    snaive = SNAIVE(Cement)
  )

# 1 (4 )
fc_cv <- fit_cv |> forecast(h = 4)
```

(b) 4단계 앞 예측 MSE 계산 및 비교

```
#      (h=4)      (MSE)
cv_mse <- fc_cv |>
  as_tibble() |>
  group_by(.id, .model) |>
  mutate(h = row_number()) |>
  filter(h == 4) |>
  inner_join(as_tibble(cement), by = "Quarter") |>
  mutate(error = Cement.y - .mean) |>
  group_by(.model) |>
  summarise(MSE = mean(error^2, na.rm = TRUE))

cv_mse |>
  arrange(MSE) |>
  knitr::kable(caption = "(b)      4      MSE ")
```

Table 8: (b) 교차 검증을 통한 4분기 앞 예측 MSE 비교

.model	MSE
ets	18873.65
snaive	19512.04

(b) 해석: CV를 통해 구한 4스텝(1년) 앞 예측 성능에서 ETS 모델이 SNaive 모델보다 소폭 낮은 MSE를 기록하여 더 우수함이 입증되었습니다. 이는 예상했던 결과입니다. 그 이유는 다음과 같습니다: 시멘트

생산량 데이터는 고정된 계절성뿐만 아니라 시간의 흐름에 따른 장기적인 트렌드 변화도 존재합니다. Seasonal Naive는 단순히 1년 전 동기 데이터를 그대로 가져오는 방식이므로 추세 변화를 전혀 반영할 수 없습니다. 반면 ETS는 과거 전체 데이터를 지수 가중하여 추세와 계절성 모두를 동적으로 학습하기 때문에 1년 뒤의 변화를 훨씬 더 정확하게 예측할 수 있습니다.

연습문제 14: ETS 모형의 범용성 한계 파악

tourism, gafa_stock, pelt 데이터에 ETS()를 적용해 보고, ETS 모형이 언제나 최선의 예측을 제공하는지 그 한계를 확인합니다.

```
# 1. tourism (      )
cat("=== 1. Tourism (      ) ===\n")
```

```
## === 1. Tourism (      ) ===
```

```
aus_trips |>
  model(ETS(Trips)) |>
  report()
```

```
## Series: Trips
## Model: ETS(A,A,A)
## Smoothing parameters:
##   alpha = 0.4495675
##   beta  = 0.04450178
##   gamma = 0.0001000075
##
## Initial states:
##   l[0]    b[0]    s[0]    s[-1]    s[-2]    s[-3]
## 21689.64 -58.46946 -125.8548 -816.3416 -324.5553 1266.752
##
## sigma^2: 699901.4
##
##      AIC      AICc      BIC
## 1436.829 1439.400 1458.267
```

```
# 2. gafa_stock - 4
# Symbol filter      reindex
cat("=== AAPL ===\n")
```

```
## === AAPL ===
```

```
gafa_stock |>
  filter(Symbol == "AAPL") |>
  model(ETS(Close)) |>
  report()
```

```
## Series: Close
## Model: NULL model
## NULL model
```

```
cat("=== AMZN ===\n")
```

```
## === AMZN ===
```

```
gafa_stock |>  
  filter(Symbol == "AMZN") |>  
  model(ETS(Close)) |>  
  report()
```

```
## Series: Close  
## Model: NULL model  
## NULL model
```

```
cat("=== GOOG ===\n")
```

```
## === GOOG ===
```

```
gafa_stock |>  
  filter(Symbol == "GOOG") |>  
  model(ETS(Close)) |>  
  report()
```

```
## Series: Close  
## Model: NULL model  
## NULL model
```

```
cat("=== META(FB) ===\n")
```

```
## === META(FB) ===
```

```
gafa_stock |>  
  filter(Symbol == "FB") |>  
  model(ETS(Close)) |>  
  report()
```

```
## Series: Close  
## Model: NULL model  
## NULL model
```

```
# 3. pelt ( )  
cat("=== 3. Pelt ( ) ===\n")
```

```
## === 3. Pelt ( ) ===
```

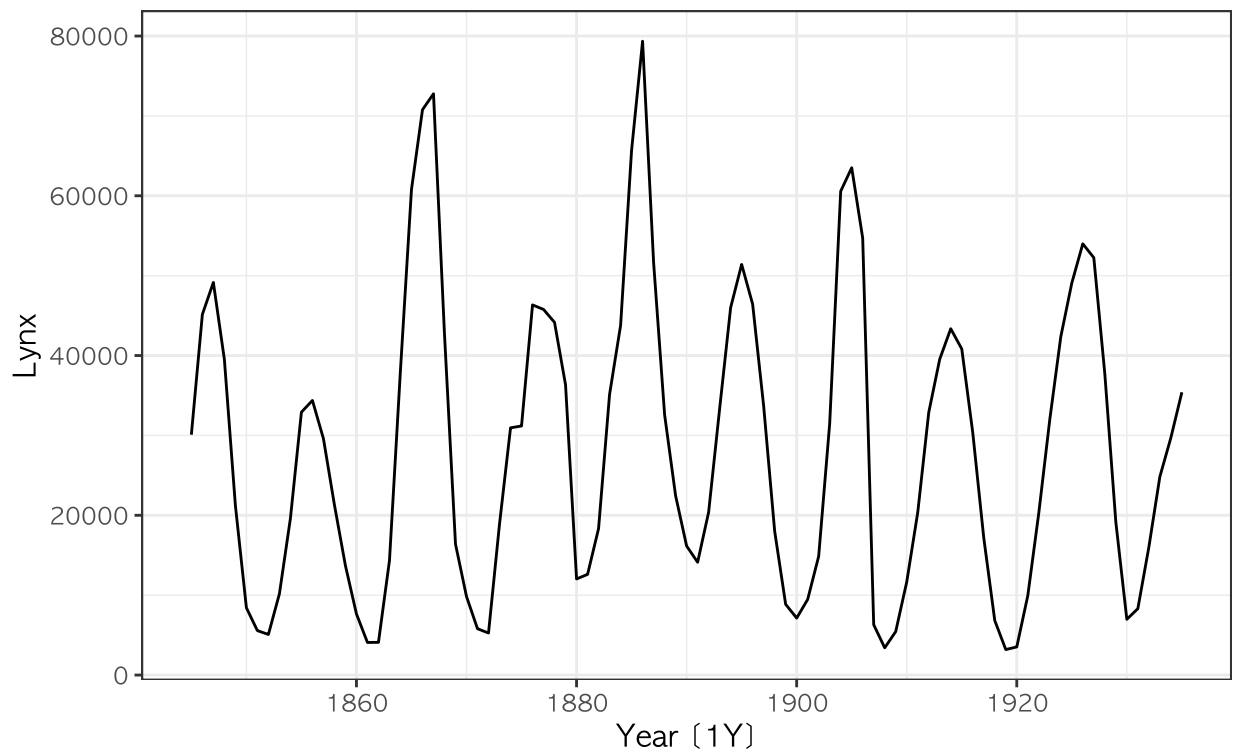
```
pelt |>  
  model(ETS(Lynx)) |>  
  report()
```

```
## Series: Lynx
## Model: ETS(A,N,N)
## Smoothing parameters:
##   alpha = 0.9998998
##
## Initial states:
##   l[0]
## 25737.12
##
## sigma^2: 162584145
##
##      AIC      AICc      BIC
## 2134.976 2135.252 2142.509
```

```
#      (ETS      )
pelt |>
  autoplot(Lynx) +
  labs(
    title = " (Lynx)      ",
    subtitle = " 10      (Cyclic) "
  ) +
  theme_bw(base_family = "AppleGothic")
```

링크스(Lynx) 연간 포획 수

약 10년 주기의 비정규 순환(Cyclic) 패턴



분석 결과 및 한계점:

tourism의 경우 ETS(A,A,A)가 선택되었으며, 추세와 계절성이 모두 존재하는 데이터 구조에 적합한 모형이 자동으로 선택된 것을 확인할 수 있습니다. gafa_stock 4종목은 모두 ETS(M,N,N)이 선택되었습니다.

주가는 수준이 높아질수록 변동성도 함께 커지는 특성이 있어 덧셈 오차(A) 대신 곱셈 오차(M)가 선택된 것으로 보이며, 추세와 계절성은 없는 랜덤 워크 구조로 판단된 결과입니다. 참고로 gafa_stock에서 Meta의 Symbol은 상장 당시 Facebook 티커인 "FB"로 저장되어 있습니다.

반면 pe1t 데이터(링크스 포획 수)에서는 ETS가 적절한 모형을 선택하지 못합니다. 링크스 개체수는 먹이사슬에 의한 약 10년 주기의 생태학적 순환(Cyclic) 패턴을 가지는데, ETS는 고정된 주기의 계절성은 처리할 수 있지만 주기와 진폭이 불규칙한 순환 구조를 명시적으로 모델링하는 메커니즘을 갖추고 있지 않습니다. 실제로 $\alpha \approx 1$ 로 추정되어 사실상 naive forecast에 가까운 모형이 적합되었으며, 이러한 순환 구조에는 AR 계열 모형이 더 적합합니다.

연습문제 16: ETS(A,N,N) 모형의 예측 분산 증명

ETS(A,N,N) 모형의 예측 분산이 $\sigma^2[1 + \alpha^2(h - 1)]$ 임을 수학적으로 보이시오.

증명:

ETS(A,N,N) 모형의 상태 공간 방정식은 다음과 같다.

1. 관측 방정식: $y_t = \ell_{t-1} + \varepsilon_t$
2. 상태 방정식: $\ell_t = \ell_{t-1} + \alpha \varepsilon_t$

단, $\varepsilon_t \stackrel{iid}{\sim} (0, \sigma^2)$. 현재 시점 T 에서 h -단계 앞 예측 분산 $Var(y_{T+h} | y_1, \dots, y_T)$ 를 구한다.

- $h = 1$:

$$y_{T+1} = \ell_T + \varepsilon_{T+1}$$

ℓ_T 는 시점 T 에서 이미 결정된 값이므로 상수 취급된다.

$$Var(y_{T+1}) = \sigma^2$$

- $h = 2$:

상태 방정식으로 $\ell_{T+1} = \ell_T + \alpha \varepsilon_{T+1}$ 을 대입하면,

$$y_{T+2} = \ell_T + \alpha \varepsilon_{T+1} + \varepsilon_{T+2}$$

ε 들이 서로 독립이므로,

$$Var(y_{T+2}) = \alpha^2 \sigma^2 + \sigma^2 = \sigma^2(1 + \alpha^2)$$

- 일반적인 $h \geq 1$:

상태 방정식을 반복 대입하면,

$$\ell_{T+h-1} = \ell_T + \alpha \sum_{j=1}^{h-1} \varepsilon_{T+j}$$

관측 방정식에 대입하면,

$$y_{T+h} = \ell_T + \alpha \varepsilon_{T+1} + \dots + \alpha \varepsilon_{T+h-1} + \varepsilon_{T+h}$$

ℓ_T 는 상수이고 ε 들은 독립이므로,

$$Var(y_{T+h} | y_1, \dots, y_T) = \underbrace{\alpha^2 \sigma^2 + \dots + \alpha^2 \sigma^2}_{(h-1)\text{개}} + \sigma^2 = \sigma^2[1 + \alpha^2(h - 1)]$$

■